Retrieval-Augmented Generation, usually shortened to RAG, combines a language model with an external knowledge base. Instead of relying only on what the model learned during training, the system retrieves relevant passages at query time and places them into the prompt. This makes the answer grounded in documents the model has never seen.

The first step of any RAG pipeline is ingestion. Documents are parsed, cleaned, and split into chunks. Chunk size is a trade-off. Small chunks retrieve precisely but lose surrounding context. Large chunks keep context but dilute relevance and cost more tokens. Most production systems start somewhere between three hundred and eight hundred characters per chunk.

Overlap between neighbouring chunks helps when a sentence is cut in half. A common setup keeps ten to twenty percent overlap, so the end of one chunk also appears at the start of the next. Without overlap, a fact split across the boundary becomes invisible to both chunks.

After chunking, every chunk is embedded into a vector. At query time the question is embedded the same way, and the nearest chunks are retrieved by cosine similarity. Pure vector search is good at meaning but weak at exact terms, so hybrid search adds keyword matching such as BM25. The two result lists are then merged and re-ranked.

Evaluation matters more than the demo. A retrieval system needs a labelled set of questions and the chunks that should answer them. With that set you can measure recall before and after a change. Without it, every improvement is a guess, and every regression goes unnoticed until a user complains.